

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	Chaitanya.K	Reg. No.:	192311450
Date:	07/10/26	Duration:	60 minutes

Code of Conduct

Chaitanya.K

192311450

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

Chaitanya.K / 192311450

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
 - Justify why the selected methodology/framework is suitable for the proposed system.
 - Explain how it supports successful project development and maintenance.
- 6. Discuss Expected Outcomes and Limitations**
- Describe the expected benefits of the proposed solution.
 - Explain how the solution addresses the identified challenges.
 - Discuss potential risks, implementation challenges, and limitations.
 - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths,	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are

CASE STUDY ANALYSIS

Self-Healing Cloud Application Failure Prediction Platform

Student: Chaitanya.K Reg. No.: 192311450

1. Understand the Problem Statement

1.1 Problem in My Own Words

Cloud applications operate through multiple containers, services, networks and dependencies. They may fail because of resource exhaustion, dependency failures, network problems, configuration errors or abnormal workloads. In a reactive approach, incident handling often begins only after users notice degradation. This increases downtime and the amount of manual troubleshooting required.

The proposed platform continuously analyses operational telemetry and looks for early indicators of failure. When abnormal behaviour or a high failure risk is detected, the system can safely initiate an approved recovery action. The action and its outcome are recorded so that engineers can audit the incident and perform root-cause analysis.

1.2 Objectives

- Collect and correlate metrics, logs and operational events.
- Detect abnormal application behaviour.
- Predict potential service failures before severe disruption.
- Trigger controlled automated recovery actions.
- Support containerized and scalable deployment.
- Provide incident timelines and root-cause analysis.
- Integrate monitoring, logging, testing and CI/CD.

1.3 Scope of the Problem

The scope includes telemetry collection and ingestion, anomaly detection, failure-risk prediction, policy-based remediation, recovery verification, incident management, audit logging, dashboards, alerts and CI/CD integration. The platform is intended to support cloud applications and their dependencies rather than replace the applications themselves.

2. Identify Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role / Responsibility
SREs	Define reliability policies, monitor incidents, review predictions, approve high-risk actions and analyse root causes.
DevOps / Platform Engineers	Maintain cloud infrastructure, containers, orchestration, deployment pipelines and integrations.
Application Developers	Fix application defects and provide useful service health indicators and telemetry.
Cloud / System Administrators	Manage environments, networking, permissions, resources and configurations.
Security Team	Define access control, secrets handling, audit and security safeguards.
Operations Managers	Track availability, incident trends, recovery performance and operational risk.
End Users / Customers	Use the application and benefit from reduced outages and faster recovery.
CI/CD and Monitoring Systems	Provide deployment events, test results, metrics, logs and alerts.

2.2 Major Challenges and Issues

- **High telemetry volume:** metrics, logs and events must be processed efficiently.
- **False predictions:** incorrect predictions can cause unnecessary remediation.

- **Unsafe automation:** a wrong recovery action can increase the impact of an incident.
- **Distributed dependencies:** the root cause may be outside the service that first appears unhealthy.
- **Changing workloads:** normal behaviour varies across time, releases and traffic levels.
- **Security:** automated recovery requires controlled privileged access.
- **Trust:** engineers need understandable reasons for predictions and actions.

3. Analyze the Current Approach

3.1 Existing System / Current Process

The case describes a mainly reactive incident-management process. Monitoring systems provide operational information and alerts, but response commonly begins after a service becomes degraded. Engineers inspect metrics and logs, diagnose the likely cause, select a recovery action, perform it and document the incident.

3.2 Strengths

- Human engineers can apply contextual judgement before high-impact actions.
- Existing monitoring and logging provide valuable evidence.
- Manual intervention can prevent an incorrect automated action.
- Incident records can support post-incident learning.

3.3 Limitations and Gaps

Gap	Effect
Response begins after degradation	Longer detection and recovery time; users may already be affected.
Manual diagnosis	Consumes engineering time and may delay recovery.
Fragmented telemetry	Metrics, logs and deployment events may not be correlated into one view.
Limited prediction	Early warning signals can be missed by fixed alerts alone.
Manual recovery	Repeated restart, scale or rollback actions are slower and less consistent.
Weak unified audit trail	Difficult to reconstruct why a decision and action occurred.

The central gap is the lack of a closed loop connecting **observation** → **prediction** → **safe decision** → **remediation** → **verification** → **audit**.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Proposed Software Solution

The proposed solution is a modular, cloud-native reliability platform. Collectors obtain telemetry from applications and dependencies. An ingestion layer validates and normalizes the data. Analytics services correlate signals and detect anomalies. A prediction component calculates failure risk. A policy engine checks safety and authorization before the remediation orchestrator executes a controlled action. A verification component checks recovery, while incident and audit services record the full event chain.

4.2 Major Functional Modules

Module	Main Functions
Telemetry Collection	Collect metrics, logs, events and dependency health information.
Ingestion & Normalization	Validate, timestamp, tag and route operational data.
Data Storage	Store recent and historical telemetry, incidents, predictions and audit records.
Anomaly Detection	Identify abnormal behaviour using rules and/or statistical or ML methods.
Failure Prediction	Calculate failure-risk scores from current and historical signals.
Policy & Decision Engine	Apply confidence, criticality, cooldown and authorization rules.

Module	Main Functions
Remediation Orchestrator	Execute approved restart, scale, traffic-shift or rollback actions.
Recovery Verification	Check health after remediation and escalate if recovery fails.
Incident & Audit	Maintain timelines, decisions, actions, overrides and outcomes.
Dashboard & Alerts	Show service health, risk, reasons, incidents and action status.
CI/CD Integration	Consume deployment/test events and support safe rollback workflows.

4.3 High-Level Architecture

Applications & Dependencies Containers • Services • Network • APIs	Telemetry Layer Collectors → Ingestion → Normalization	Analytics Layer Correlation → Anomaly Detection → Prediction
Decision Layer Risk → Policy/Safety Checks → Approval	Action Layer Remediation → Cloud/Container APIs	Verification Health Check → Incident Record → Feedback

Safety principle: prediction must not directly equal action. A policy engine should verify confidence, service criticality, cooldown periods, recent actions and authorization before remediation.

4.4 Functional Requirements

ID	Requirement
FR1	Collect metrics, logs and operational events from supported applications and dependencies.
FR2	Normalize and correlate telemetry using service, container, host and time information.
FR3	Detect abnormal application behaviour.
FR4	Calculate a failure-risk score with interpretable contributing signals.
FR5	Evaluate remediation policies before automated actions.
FR6	Execute only authorized and pre-approved remediation actions.
FR7	Verify service health after remediation.
FR8	Create incident timelines covering detection, prediction, decision, action and verification.
FR9	Provide dashboards and alerts for operators.
FR10	Support CI/CD and monitoring integrations through defined interfaces.

4.5 Non-Functional Requirements

Quality	Requirement / Design Target
Reliability	Avoid single points of failure and preserve critical incident/audit information.
Scalability	Scale ingestion and analytics horizontally as service and event volume grows.
Performance	Process critical signals quickly enough to provide useful early warning.
Security	Use least privilege, strong authentication, authorization, encryption and protected secrets.
Usability	Present health, risk, reasons, actions and timelines clearly to operators.
Maintainability	Use clear interfaces, automated tests, versioned configurations and documentation.
Auditability	Record actor/service, time, policy, reason, action and outcome for automated actions.
Availability	Keep monitoring and recovery services available despite individual application failures.

4.6 Software Engineering Principles

Principle	Application
Modularity	Separate collection, ingestion, analytics, prediction, policy, remediation, verification and audit components.
Scalability	Use independently scalable services and queue-based ingestion where appropriate.
Maintainability	Use version control, API contracts, automated tests, configuration management and documentation.
Reliability	Use health checks, bounded retries, timeouts, circuit breakers, redundancy and recovery verification.
Usability	Provide an operator dashboard with understandable risk levels and action status.
Security	Apply RBAC, least privilege, encryption, secret management and secure APIs.
Observability	Instrument the platform itself with metrics, logs and traces.
Testability	Use unit, integration, system, fault-injection and recovery testing.

5. Justify the Chosen Methodology/Framework

5.1 Selected Approach: Agile Scrum with DevOps Practices

Agile Scrum is appropriate because the platform contains complex and evolving areas such as telemetry integration, prediction logic, remediation policies, dashboards and cloud integrations. Requirements can change as engineers learn from incidents and evaluate prediction accuracy. Scrum supports incremental delivery through short sprints, while DevOps practices support continuous integration, automated testing, deployment and operational feedback.

5.2 Why This Approach Fits

- **Incremental delivery:** telemetry and dashboards can be built before advanced prediction and automated recovery.
- **Stakeholder feedback:** SREs and DevOps engineers can refine thresholds, policies and usability every sprint.
- **Risk reduction:** high-impact automation can remain behind feature flags until sufficiently tested.
- **Continuous testing:** CI/CD can run unit, integration, security and regression tests.
- **Operational feedback:** incident and recovery results can improve rules, models and remediation policies.
- **Maintainability:** security, observability and technical debt remain visible backlog items.

5.3 Example Sprint Plan

Sprint	Main Deliverables
1	Architecture, service inventory, telemetry schema, authentication and basic dashboard.
2	Metrics/log/event ingestion, storage and correlation.
3	Anomaly detection, alerting and incident timeline.
4	Failure-risk prediction, confidence scoring and explanation.
5	Policy engine, safety checks and controlled remediation in a test environment.
6	Recovery verification, CI/CD integration, security hardening and end-to-end testing.
7+	Pilot rollout, feedback, tuning, performance improvements and production expansion.

5.4 Maintenance Strategy

After deployment, the same DevOps feedback cycle should be used for maintenance. Changes should pass automated tests and security checks. Prediction rules/models should be versioned and evaluated against historical incidents. Remediation policies should be reviewed after significant incidents, and unsafe or repeatedly ineffective actions should be disabled until corrected. The self-healing platform itself must also be monitored.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Benefits

- Detect potential failures before major service disruption.

- Reduce mean time to detect and mean time to recover.
- Improve availability and resilience of cloud applications.
- Reduce repetitive manual incident-response effort.
- Improve operational visibility by correlating metrics, logs and events.
- Provide consistent and auditable recovery workflows.
- Create a foundation for progressively more capable self-healing systems.

6.2 How the Solution Addresses Challenges

Challenge	Response
Telemetry overload	Normalize, tag, aggregate and route telemetry through scalable pipelines.
Slow diagnosis	Correlate signals automatically and provide a unified incident timeline.
Late response	Use anomaly detection and failure prediction for early warning.
Unsafe automation	Use policy checks, confidence thresholds, cooldowns, approvals and recovery verification.
Distributed causes	Correlate application, dependency, network and deployment events.
Security risk	Use least privilege, RBAC, secure secrets and complete audit records.

6.3 Risks and Implementation Challenges

- **False positives/negatives:** prediction quality may initially be imperfect.
- **Automation blast radius:** an incorrect policy could affect healthy services.
- **Data quality:** missing or inconsistent telemetry can reduce prediction accuracy.
- **Model drift:** application behaviour changes after releases and workload changes.
- **Integration complexity:** cloud and monitoring platforms expose different APIs.
- **Cost:** telemetry storage and analytics can become expensive without retention controls.
- **Organizational adoption:** engineers need evidence that automation is safe and explainable.

6.4 Limitations

The system cannot guarantee that every failure will be predicted or automatically repaired. Novel failures with no useful telemetry pattern may remain difficult to predict. Some incidents require human judgement, especially where recovery could cause data loss or business impact. Automated remediation also depends on the permissions and APIs exposed by the underlying cloud and container platform. A graduated automation model is therefore recommended: observe first, recommend next, automate low-risk actions and require approval for high-impact actions.

6.5 Future Enhancements

- Advanced time-series and machine-learning models for multi-signal failure prediction.
- Explainable-AI features showing the signals contributing to each prediction.
- Service dependency graphs for improved root-cause localization.
- Automated canary analysis and safer deployment rollback decisions.
- Feedback mechanisms to learn which remediation actions are most effective.
- A policy simulation/test environment for recovery validation before production.
- Multi-cloud support and standardized integrations.
- Reliability scorecards and trend analytics for engineering teams.

Conclusion

The Self-Healing Cloud Application Failure Prediction Platform addresses a major weakness of reactive cloud operations by creating a controlled closed-loop reliability process. It combines telemetry collection, anomaly detection, failure prediction, policy-based decision making, automated remediation, recovery verification and auditability. A

modular architecture supports scalability and maintenance, while security and safety gates reduce automation risk. Agile Scrum with DevOps provides an appropriate development approach because the system can be delivered incrementally, tested continuously and improved using operational feedback. With careful validation and gradual automation, the solution can reduce downtime, recovery effort and operational complexity while improving cloud application reliability.

Submission Checklist

- Official first two assessment pages retained at the beginning.
- Student name and registration number entered.
- Signature area left blank for handwritten signature.
- All six required analysis areas addressed.
- Official Rubrics page retained as the final page.

	weaknesses, and identifies all gaps.			largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25